Linear-Time Tape Transposition Requires Storage Codes with Rates Approaching One

Hans Brende GPT-6 Astra (OpenAI)

November 2026

Abstract

Suppose a deterministic multitape machine transposes square matrices in linear time at arbitrarily large dimensions d, with entries of $\max(1, \lceil \log_2 d \rceil)$ bits and head moves that may depend on the input. For every sufficiently large incidence-degree cap Δ we obtain a family of binary storage codes whose minimum distance between paired input and recovery ports tends to infinity. Its limiting lower rate is at least

$$1 - O_{M,c}\left(\left(\frac{\log_2 \log_2 \log_2 \Delta}{\log_2 \log_2 \Delta}\right)^{1/(4k^2+1)}\right)$$

for k work tapes and time coefficient c. The transfer also holds on jointly growing dimensions and entry widths when the rate of the successful input set tends to one. Counting the joint records of whole simulator batches gives the same exponent for codes on the original bit positions, recovered at their actual append times. On any nonempty successful set G of L-bit inputs, the fraction of inputs whose fibre rate falls more than this allowance plus $u \geq 0$ below $\log_2 |G|/L$ is at most 2^{-uL}. For these codes, bounded-overlap patches face a further restriction. The sum of the information their permitted helpers supply about individual separated targets is at most twice the overlap times the total correlation of the input. All numbered results are proved in Lean. An upper bound on the attainable storage rates at the required scale would exclude linear-time transposition.

Contributions. Hans Brende chose the problem and directed the research. GPT-6 Astra developed the arguments, wrote the manuscript and formalized the proofs in Lean.

Contents

1 Introduction **2**
2 Tape computations and recovery relations **3**
3 The rate forced by a packet computation **4**
4 A degree gap sufficient for exponential failure **7**
5 A bit simulator with few motion traces **7**
6 Typical codes on physical bit positions **9**
7 What can overlapping patches predict? **12**
8 What remains open **15**
A Formal statements and verification **16**
B Index of Lean declarations **17**

1 Introduction

The entries of a matrix's first column are a row apart on the input tape and consecutive in the output. A transposer may store coded information as it works, and its head movements may depend on what it has read. If we fix those movements, we can trace the possible dependencies of an output bit. Fixing the movements also restricts the input. The question is how much information survives the restriction.

For a fixed tape trajectory, a depth-reduction theorem supplies a small set of steps whose values can be revealed so that the remaining dependency paths have bounded length [2, Corollary O]. On the inputs sharing both the trajectory and the revealed values, each output can then be recovered from a bounded set of input coordinates. The recovered coordinate and its input copy can be far apart in the incidence graph of that relation. Such separation is compatible with binary storage codes of rate above one half [6, Corollary A′]. A linear-time transposer would require rates arbitrarily close to one.

Theorem 3.1 gives the degree dependence of this requirement. For every sufficiently large degree cap it produces a family with rates bounded below while its minimum paired distance tends to infinity. The coordinates are bits of packets. The degree dependence is explicit. A coding upper bound with a larger deficit would rule out the assumed machine. Corollary 4.1 makes the comparison precise, including the success probability when the machine is correct only on some inputs.

A second question concerns the individual bit positions of a physical machine. Theorem 5.1 groups source transitions into batches but executes each head move and each append on an ordinary bit machine. Theorem 6.2 uses its actual append times. The packet graph connects every bit of a permitted helper packet to every bit of its target packet. The native graph follows each physical append backwards through the simulator. On a set G of correctly transposed inputs, the native theorem bounds the input mass in fibres whose rate falls appreciably below $\log_2 |G|/L$. Each fibre keeps nearly all of its trace's original bit positions. Revealing complete simulator batches and counting their joint record support gives the same rate-loss exponent as the packet theorem.

Finally, Section 7 asks how much a local set of helpers can tell us about the bit it is meant to recover. The answer accounts separately for a bit's own bias and the information supplied by its helpers. A fixed advantage over chance on a positive fraction of separated targets requires increasingly large patch overlap as the code rate approaches one. The outstanding question is whether a linear-time tape computation would have to provide such an advantage at smaller overlap.

Relation to earlier work. The finite-control simulation of Hartmanis and Stearns evaluates consecutive tape operations from a bounded neighbourhood [11, Theorem 2]. Hopcroft, Paul and Valiant synchronize tape-block crossings [12, §2]. Williams describes the computation graph of an aligned machine using a constant number of movement bits per block [13, §§2–3]. Here the sequence of batch descriptors determines the in-batch append flags and when the terminal drain starts. The common output length then fixes the number of bits appended during the drain. Earlier in this series, an external-memory simulation fixed an event word at the scale of entire rows [4, Theorem EM]. Our batch size is fixed before the matrix dimension grows. Conditional entropy as a measure of imperfect local recovery appears in earlier work on recoverable systems [10, §IV]. Our prediction bound uses the separated-port argument of [7, Lemma 4.1] and the soft-recovery inequality for nested elimination [9, Theorem S].

Formal proofs. The Lean declarations named with each numbered result are its proofs. The English outlines explain the constructions, estimates and choices of parameters. Appendix A gives the verification procedure. The declaration index identifies the corresponding source modules.

2 Tape computations and recovery relations

The machine and its task. We use the deterministic multitape model of the first paper in the series [3]. The machine has finite control, a two-way read-only input tape, $k \geq 0$ two-way work tapes over a fixed finite alphabet, and an append-only binary output. All heads start at zero and move by at most one cell per transition. A transition writes at the scanned work cells and appends at most one bit. Reading beyond the input gives an end marker. The *full motion trace* records the head directions and whether a bit was appended at each transition, together with termination. The next control state, written symbols and appended bit belong to the step value [4, §2].

A $d \times d$ matrix with $b \geq 1$ bits per entry occupies $L = d^2 b$ bits in row-major order. For $0 \leq i, j < d$ and $0 \leq t < b$, the input bit at address $q = (id + j)b + t$ must be appended at output position $\mathrm{tpos}(q) = (jd + i)b + t$. Fix a machine M and a time coefficient c. Write $G_{d,b}$ for the inputs on which M halts within $c(L + 1)$ steps with this output. At the canonical width $b(d) = \max(1, \lceil \log_2 d \rceil)$, statement (T) asks for a single M, c with $G_{d,b(d)} = \{0,1\}^L$ for every positive d.

A code and its paired distances. Let V be a finite nonempty coordinate set and $\mathcal{A}$ a finite alphabet of size at least two. A relation N assigns to each $q \in V$ a set $N(q) \subseteq V$ of permitted helpers. A nonempty set $C \subseteq \mathcal{A}^V$ is a *storage code* for N if two words of C agreeing on $N(q)$ agree at q, for every q. Its rate is $\log_{|\mathcal{A}|} |C|/|V|$. No linearity is assumed.

The incidence graph has an input port L_v and a recovery port R_v for each v. Join L_v to R_q when $v \in N(q)$. The *paired distance* of q is the undirected graph distance from L_q to R_q, with value ∞ for disconnected ports. For example, take two coordinates that store the same bit and let each use its partner as its only helper. The two incidence edges are disjoint. Both paired distances are infinite, although each coordinate is recovered exactly. This graph measurement concerns the permitted observations at the two ports. The example has rate $1/2$ [7, §3].

A binary family (V_i, N_i, C_i) is *admissible at cap* Δ if every port has degree at most Δ, $|V_i| \to \infty$, and for each fixed integer radius r every paired distance exceeds r for all sufficiently large i. Let $R(\Delta)$ be the supremum, over these families, of their limiting upper rates. Our constructions give lower bounds on the limiting *lower* rate. For large Δ, write

$$p = 4k^2 + 1, \qquad f_j(\Delta) = \left(\frac{\log_2 \log_2 \log_2 \Delta}{\log_2 \log_2 \Delta} \right)^{1/j} \quad (j > 0).$$

Both the packet and native bounds below use $j = p$.

How a tape run gives a recovery relation. Freeze a motion trace. In the diagram of step dependencies, a step has at most $k + 1$ earlier predecessors, one for the previous control step and one for the last visit to the scanned cell on each work tape. Missing predecessors give initial values. The input head supplies the bit at its current address, or the end marker. Choose a set Γ of steps and reveal their full values. This collection of observations is called a *genie*. Start at the append of output bit $\mathrm{tpos}(q)$ and follow the predecessor edges backwards, requiring every vertex after the root to lie outside Γ. The root itself may be revealed. The input addresses encountered form the *cone* of that append. Once the trace and the revealed values have been fixed, agreement on these input bits forces agreement at the append, by induction on time. On correctly transposed inputs, it therefore forces agreement at q. This is the recovery relation used at physical bit scale. The same recursion with packet inputs and packet records will be used in Section 3 [4, §4].

For k work walks with $T \leq C(n + 1)$ steps, $C \geq 1$, $n \geq 1$, a cut budget $\varepsilon > 0$ and records of width $\beta \geq 1$, the bounded-stack theorem gives a cut with $\beta|\Gamma| \leq \varepsilon n$. Each permitted path then has

at most $D+1$ vertices, where $D = \mathrm{genieDepth}(k, \beta, C, \varepsilon)$ is independent of n [2, Corollary O(a)]. Here n counts input units. A unit is one packet in Section 3 and one physical bit in Section 6. In both applications the revealed record values are charged in bits.

3 The rate forced by a packet computation

Theorem 3.1 (Theorem A, quantitative packet transfer)**.** `Lean`. `TapeStorageRates.rate_witnesses_of_growing_widths`, `TapeStorageRates.rate_witnesses_of_near_full`, `TapeStorageRates.rate_witnesses_of_full` in `TapeStorageRates/Main.lean`.

Fix a deterministic machine M with k work tapes and a time coefficient $c > 0$. Suppose that for every $n \geq 1$ and $\eta > 0$ there are $d, b \geq n$ for which

$$G_{d,b} \neq \varnothing, \qquad \log_2 |G_{d,b}| \geq (1-\eta) d^2 b.$$

There is a constant $K = K(M, c) > 0$ such that for every sufficiently large integer Δ some admissible binary family at cap Δ satisfies

$$\liminf_{i \to \infty} \frac{\log_2 |C_i|}{|V_i|} \;\geq\; 1 - K f_{4k^2+1}(\Delta).$$

Consequently $R(\Delta) \geq 1 - K f_{4k^2+1}(\Delta)$ for all sufficiently large Δ, and $R(\Delta) \to 1$.

Outline of the formal proof. Fix s bits per packet. On a sufficiently large successful instance, conditioning on the incomplete ends of entries, the packet schedule and the revealed steps leaves a code with a deficit of $O_{M,c}(1)$ bits per original packet. The trimming estimate below retains at least half the packets at the chosen cap. Expanding those packets into bits gives a rate loss of $O_{M,c}(1/s)$. The required degree cap Δ_s satisfies $\log_2 \log_2 \Delta_s = O_{M,c}(s^p \log_2(s+1))$. Inverting it gives the claimed rate. The following two subsections establish the finite estimates and make the order of the choices explicit. □

Both thresholds $d, b \geq n$ apply to the same success set. In particular, the hypothesis holds if M succeeds on $2^{L-o(L)}$ inputs along a sequence of canonical dimensions tending to infinity. A machine satisfying (T) succeeds on all 2^L inputs at those dimensions. The family in the conclusion may depend on Δ. Once the cap is fixed, its length and its minimum paired distance grow together.

3.1 From successful inputs to a common packet schedule

Fix a packet size $s \geq 1$. Use the hypothesis of Theorem 3.1 with $\eta = 1/(2s)$. We can choose d as large as needed with $b \geq s$ and a nonempty success set $G = G_{d,b}$ of deficit $h = L - \log_2 |G| \leq L/(2s)$. Write $b = as + r$ with $a \geq 1$ and $0 \leq r < s$. Each matrix entry consists of a complete s-bit packets followed by r tail bits. There are $n = d^2 a$ complete packets. The packet at (i, j, ℓ) is demanded at (j, i, ℓ) for $0 \leq \ell < a$. Since $L = sn + rd^2 \leq 2sn$, we have $h \leq n$.

Partition G by the entire string of rd^2 tail bits. Some fixed assignment z leaves a set F_z of successful packet arrays with

$$|F_z| \geq |G|\, 2^{-rd^2} = 2^{sn-h}.$$

Thus fixing the tails preserves the bound h on the absolute deficit, with sn as the new ambient bit count. Throughout this computation the tail of an entry is the fixed string specified by its address in z.

The auxiliary input tape uses one cell per packet. The last cell of each entry holds its s payload bits followed by its fixed tail, so its length in the original input is $s+r$. All other in-range cells have length s. Each cell outside the input represents s end markers, and each work cell holds s ordinary work symbols. Starting from any offset in a cell, s source transitions can visit only that cell and its two neighbours.

One batch evaluates these transitions in eleven packet ticks. The input and work heads move one cell left and read the left, centre and right cells in succession. A stationary tick evaluates the batch on these windows. The heads traverse the windows in reverse to write back the three work cells, return to the old centres, and move by at most one cell to the new centres. Other work writes preserve the scanned symbols. The final stationary tick may append one packet. At the start of a batch the pending output is shorter than its next cell. The batch adds at most s raw output bits, so it completes at most one cell. On completion, emit its first s bits, pass over its tail, and keep the suffix for the next cell. Every successful run emits exactly the n payload packets in transposed order. The public cell lengths and tail values belong to this auxiliary computation. Section 5 will return to ordinary bit tapes.

Use $R = \lfloor \lfloor c(L+1) \rfloor / s \rfloor + 1$ batches on every input, keeping the source configuration fixed after halt. For $J = 2\lceil c \rceil + 1$, the bounds $R \le J(n+1)$ and $T = 11R \le 11J(n+1)$ are uniform in s, d, b. Within each batch the varying part of the schedule consists of the $k+1$ displacements to the new centres and the packet-append flag. These give at most $(2 \cdot 3^{k+1})^R$ schedule words. Hence for an integer $A_0 = A_0(k, c)$ some fixed schedule has a class W satisfying

$$\log_2 |W| \;\ge\; sn - h - A_0 n \;\ge\; sn - (A_0+1)n.$$

The compiler further records the finite control, offsets, windows, pending output, work writes and any output packet. They fit in $\beta_s = O_M(s)$ bits per tick. On the fixed schedule the input address, cell type, tick within the batch, and count of previously appended packets are public. The record at a tick is determined by its input packet, the preceding control record, and the records from the last visits to the scanned work cells. Initial values and exterior input symbols are fixed. This locality and the output at each designated append tick are proved by `TapeStorageRates.PaidClasses.compiler`.

3.2 Extracting and separating a code

Call W a *frozen packet class* on $n = d^2 a$ packets if its nonempty set of packet arrays has common unit walks for the input head, the k work heads and the output position, and its tick records obey the locality rule above. It also has a designated tick before the deadline for each demanded packet. At that tick the output position is the packet's transposed address and the record determines the packet on W. In the compiled class this is its append tick.

Proposition 3.2 (Retention at a chosen degree threshold)**.** `Lean`.
`TapeStorageRates.packet_code_at_threshold` in `TapeStorageRates/PacketFamily.lean`.

Fix integers $k, A \ge 0$, $s, \beta \ge 1$ and a time coefficient $c_0 \ge 1$. Let $D = \mathrm{genieDepth}(k, \beta, c_0, 1)$ be the depth bound of the bounded-stack walk theorem, and put $B = \sum_{j=0}^{D} (k+1)^j$. Fix a degree threshold $\theta \ge 0$, a radius $u \ge 0$, both integers, and a real α with $0 < \alpha < 1 - B/(\theta+1)$. For all sufficiently large d, every $a \ge 1$ and every frozen class W on $n = d^2 a$ packets with β-bit records, at most $c_0(n+1)$ ticks and $\log_2 |W| \ge sn - An$, there is an integer m with $\alpha n \le m \le n$ and a binary storage code on sm coordinates. Its degree is at most $s \max(B, \theta)$, every paired distance exceeds u, and its rate is at least $1 - (A+1)/(\alpha s)$.

Outline of the formal proof. Point each tick back to its control and work predecessors. The walk theorem supplies a set Γ of ticks with $\beta|\Gamma| \le n$. On a dependency path starting at an output tick

and avoiding Γ after its first vertex, the depth bound is D. Fix a most frequent joint value for the records at Γ. This costs at most n bits in log cardinality. For a source packet q, let $N(q)$ contain the input addresses reached by these paths from its designated output tick. Two arrays in the resulting fibre that agree on $N(q)$ have the same output record there. Indeed the revealed records already agree, and the locality rule gives agreement at the remaining ticks by induction in time. Thus $N(q)$ recovers q. This application uses the whole-step walk cut of [2, Corollary O(a)] and the common-schedule recovery argument of [1, proof of Corollary 6.4].

Each tick has at most $k+1$ predecessors. Consequently $|N(q)| \le B$, and the sum of all input-port degrees is at most Bn. Remove both ports of each coordinate whose input degree exceeds θ. There are at most $Bn/(\theta+1)$ such coordinates. In the induced graph the maximum degree is at most $\max(B,\theta)$. The paired-path packing theorem bounds the number of remaining matched pairs at distance at most u by $O_{k,c_0,D,\theta,u}(n/\log_2 d)$, uniformly in a. At bounded degree, a positive density of short paths would yield many vertex-disjoint ones. Following their dependency edges forwards and backwards through the walks would transport the corresponding input coordinates to their transposed positions. The row-entropy transport bound limits the number that can be served in $O(n)$ ticks. This is where the transposed addresses enter the estimate. The formal packing bound uses the walks, the cut and the positions of the transposed demands [7, Theorem 3.3 and Remark 3.9]. At block scale one, the Lean estimate `MergeDepth.skel_paired_packing` underlying [4, Proposition 4.11(a)] also permits $k = 0$. The strict inequality defining α leaves room to remove these short pairs and retain $m \ge \alpha n$. Every surviving path was already a path before deletion.

Fix the omitted packets to common values when projecting the code through the two deletions. The total cost is at most $s(n-m)$ bits, one s-bit payment for each omitted coordinate. Its former occurrences in the recovery rules are now constants [7, Corollary 3.8]. The projected code has log cardinality at least $sm-(A+1)n$. Replace every surviving packet by its s bits and each incidence by all s^2 incidences between the corresponding copies. The bits of the helper packets determine each demanded bit. Expansion preserves code cardinality, multiplies the degree bound by s, and sends every paired path back to a packet walk of the same length when its bit indices are forgotten [6, Lemma 8.1]. Dividing the cardinality bound by sm gives the rate in the statement. □

Completion of the proof of Theorem A. Apply Proposition 3.2 with $c_0 = 11J$, $\beta = \beta_s$ and $A = A_0+1$ from the schedule count. Taking $\theta = 2B$ permits $\alpha = 1/2$, since $B/(2B+1) < 1/2$. Thus the cap is $\Delta_s = 2sB$ and the rate loss is at most $2(A+1)/s$. At cap $10sB$, taking $\theta = 10B$ permits a retained fraction of at least $9/10$. For each successive radius u, take a qualifying d, b large enough for the proposition and with $d \ge 2(u+1)$. Their lengths tend to infinity, and the uth code has every paired distance greater than u.

It remains to fit Δ_s below a prescribed Δ. Write $x = 4\beta_s c_0 = O_{M,c}(s)$. In the walk cut the full depth is $D = K_0^{P_0}\lceil x \rceil$, with its radix K_0 and digit count P_0 taken at cut fraction $1/x$ [2, Definition 7.1]. For fixed k the parameter bounds give $\log_2 K_0 = O_k(x)$ and $P_0 = O_k(x^{4k^2}\log_2(x+1))$. Since $B \le (k+2)^{D+1}$, these inequalities yield

$$\log_2\log_2 \Delta_s = O_{M,c}\Big(s^{\,4k^2+1}\log_2(s+1)\Big).$$

With $y = \log_2\log_2\Delta$ and $p = 4k^2+1$, choose s as the integer part of a sufficiently small fixed multiple of $(y/\log_2 y)^{1/p}$. For all sufficiently large Δ this gives $\Delta_s \le \Delta$ and $1/s = O_{M,c}(f_p(\Delta))$. The family at that s proves the limiting lower rate bound. The supremum $R(\Delta)$ is at least this family's limiting upper rate, and every binary code has rate at most one. This proves the convergence asserted in Theorem 3.1. □

4 A degree gap sufficient for exponential failure

Theorem A gives a test for an impossibility proof. At a degree cap where all admissible families have a rate gap ρ, a putative transposer must pay it from its deficit allowance $Kf_p(\Delta)$. The machine constant K is fixed before Δ is chosen.

Corollary 4.1 (A sufficient gap at a sequence of caps)**.** `Lean`. `TapeStorageRates.exponentially_small_at_widths_of_sparse_gap` in `TapeStorageRates/Main.lean`.

Fix a tape cap $k \geq 0$ and put $p = 4k^2 + 1$. Suppose there are integer caps $\Delta_j \to \infty$ and real numbers ρ_j such that $\rho_j / f_p(\Delta_j) \to \infty$. At each Δ_j, assume that every admissible family has limiting lower rate at most $1 - \rho_j$. For each fixed machine M with at most k work tapes and each $c > 0$, there are $\gamma > 0$ and n_0 such that

$$\frac{|G_{d,b}|}{2^{d^2 b}} \leq 2^{-\gamma d^2 b} \qquad (d \geq n_0,\ b \geq n_0).$$

In particular the same exponential upper bound holds at every sufficiently large canonical dimension, and (T) fails for such machines.

Outline of the formal proof. If the joint near-full success hypothesis of Theorem 3.1 held, each sufficiently large cap would admit a family of limiting lower rate at least $1 - Kf_p(\Delta)$. A machine with fewer than k tapes also satisfies this bound, since increasing the tape count increases the deficit allowance at large caps. Choose j with Δ_j large enough for the lower bound and $\rho_j > Kf_p(\Delta_j)$. The required family violates the assumed upper bound at that same cap. Negating the near-full success hypothesis now gives some $\gamma > 0$ and n_0 such that, throughout $d, b \geq n_0$, each nonempty success set has $\log_2 |G_{d,b}| \leq (1 - \gamma) d^2 b$. Empty success sets satisfy the probability bound as well. Finally $b(d) \to \infty$ with d. □

For example, suppose $R(\Delta) \leq 1 - \rho(\Delta)$ at every sufficiently large Δ, with $\rho(\Delta) \geq 0$. Writing $y = \log_2 \log_2 \Delta$, a gap $\rho(\Delta) \geq c_1 / y^a$ for a constant $c_1 > 0$ and $0 < a < 1/p$ suffices for the fixed tape cap k. If $\limsup_{\Delta \to \infty} \rho(\Delta) y^a = \infty$ for every $a > 0$, the exclusion applies to every fixed finite tape count. Indeed $y^{1/(2p)} f_p(\Delta) \to 0$. A gap bounded below by a positive constant times $1/\log_2 y$ meets this all-tapes condition. The elementary gap $1/(\Delta + 1)$ obtained by ordering coordinates has ratio to $f_p(\Delta)$ tending to zero [6, open problems]. The slowly decreasing gap criterion for oblivious machines appears after Corollary O in [2]. The power comparison and the elementary-gap comparison are checked in `TapeStorageRates.ratio_tendsto_of_power_gap` and `TapeStorageRates.elementary_gap_ratio_tendsto_zero`. The coding upper bound at the scale required for Corollary 4.1 remains open.

5 A bit simulator with few motion traces

We now keep the matrix in its original binary layout. The batch size controls how often a simulator reveals an input-dependent decision in its head motions. The purpose is to leave a large set of inputs on which the physical cone at each append has the same meaning.

Theorem 5.1 (Theorem S, native trace count)**.** `Lean`.
`TapeStorageRates.Native.same_partial_function`,
`TapeStorageRates.Native.source_support_fixed`, `TapeStorageRates.Native.source_support` in
`TapeStorageRates/NativeCompiler.lean`.

Fix a deterministic machine M with k work tapes and an integer $s \geq 1$. There is a machine N_s with the same number of tapes and the same work alphabet that computes the same partial function. On a halting run of t successful transitions of M, its running time is exactly

$$(10s + 1)(\lfloor t/s \rfloor + 1) + s.$$

Its finite control can be encoded in $O_M(s)$ bits. On any set of halting inputs with $t \leq U$ and common output length, its number of distinct full motion traces is at most

$$(1 + H_k)^{\lfloor U/s \rfloor + 1}, \qquad H_k = 2 \cdot 3^{k+1}.$$

If the output lengths vary, multiply this bound by s.

Outline of the formal proof. At the start of a batch the physical head on tape j sits at an anchor $p_j \in s\mathbb{Z}$. Its simulated head is at $p_j + r_j$, where $0 \leq r_j < s$. The simulated work tapes agree with the physical ones. The control holds the offsets, simulated state and a word of fewer than s pending output bits. The physical output followed by this word is the simulated output so far. All anchors and offsets start at zero.

Move the heads left s cells. In the next $3s$ ticks read the windows $[p_j - s, p_j + 2s - 1]$, moving right except on the final read. In one stationary tick a finite table simulates up to s transitions on these windows and stops if it detects halt. Every visited cell and the final position are inside the scanned window, since the starting offset is in $[0, s - 1]$. Each ending offset relative to the old anchor has a unique expression $se_j + r'_j$, where $e_j \in \{-1, 0, 1\}$ and $0 \leq r'_j < s$. The table also joins the new output bits to the pending word. Their total length is at most $2s - 1$. If there are at least s bits, schedule the first s for emission.

Write the updated work windows back from right to left in $3s$ ticks, moving left except on the last write. Every head follows this sweep, including the input head. Return each head to its old anchor in s rightward ticks. In the next s ticks each takes direction e_j, reaching its new anchor. Hold the heads still for s ticks and append the scheduled bits, or idle throughout if none were scheduled. The pending word is the unprinted suffix. These phases preserve the boundary conditions. Work writes outside the writeback phase preserve the scanned symbol.

In Lean, `TapeStorageRates.Native.simulates_halt` proves the exact clock and equality of completed outputs. Each batch costs $10s + 1$ ticks. If a batch discovers halt, drain the pending word during s final stationary ticks, idling after its last bit, then halt. When s divides t, the extra batch discovers halt before simulating any further transition. An infinite simulated run executes infinitely many batches. The windows, pending output, offsets, counters and finite state admit $O_M(s)$ bits of control by `TapeStorageRates.Native.control_bits`. For fixed M, s the update on this finite set is a transition table.

A batch's $k + 1$ directions e_j and its full-word emission flag determine all its native moves and append flags. Pad the sequence of batch descriptors, using a distinct symbol, to length $\lfloor U/s \rfloor + 1$. Its unpadded length determines when to drain and halt. At a common output length ℓ the drain emits exactly $\ell \bmod s$ bits. At variable lengths this count takes at most s values. This gives the claimed trace bounds. □

For inputs of length L with $t \leq c(L + 1)$ and common output length, the limiting upper value of $L^{-1} \log_2 |\operatorname{supp}(\text{trace})|$ is at most $c \log_2(1 + H_k)/s$. The limsup inequality is `TapeStorageRates.`

`Native.trace_support_rate_limsup`. Thus any prescribed positive trace rate selects one simulator before L grows.

The time coefficient can also be made uniform in s on all halting inputs. Simulate the first $2s$ successful transitions directly, stopping if M halts earlier. In the batched phase, use the current physical head positions as grid origins and start with empty pending output. For $t < 2s$ the total cost is t. Otherwise it is

$$3s + (10s + 1)(\lfloor t/s \rfloor - 1) \leq 11t.$$

The variant uses $O_M(s)$ bits of control. Its prefix adds $O_k(s)$ bits to the trace description. The prefix flags specify how many bits were already emitted there, so a fixed total output length still determines the length of the final drain. The formal bounds are `TapeStorageRates.Prefix.simulates_halt`, `TapeStorageRates.Prefix.time_le_eleven`, `TapeStorageRates.Prefix.control_bits`, and `TapeStorageRates.Prefix.source_support`.

For this compiler the order $1/s$ can be attained, as proved in `TapeStorageRates.Native.copy_trace_card`. Consider a copying machine that advances a dummy work head exactly on a 1, and advances the input head after each copied bit. Independently choose each of the first $\lfloor L/s \rfloor$ aligned input blocks to be all zeros or all ones, fixing the suffix. In each batch the dummy head stays at its anchor or advances to the next anchor. These choices give $2^{\lfloor L/s \rfloor}$ different simulator traces. The example measures the compiler's trace count on a specified machine. Copying also admits an oblivious implementation.

6 Typical codes on physical bit positions

Run the simulator N_s of Theorem 5.1 on a nonempty set $G \subseteq G_{d,b}$. We will count inputs in G with equal weight. The construction has to keep track of which physical bit positions survive, because the most frequently read bits can differ between traces.

6.1 Counting the cells of a frozen trace

For each attained full trace τ, choose one set of revealed steps Γ_τ. Root the cone for input coordinate q at the step that appends output bit $\mathrm{tpos}(q)$. Delete both ports of each coordinate whose input port has excessive degree. In the induced graph delete both ports of every remaining coordinate whose paired distance is at most the chosen radius. Denote the retained input positions by U_τ and set $m_\tau = |U_\tau|$. The choices of steps and positions are common to the whole trace. The existence of such cuts with bounded depth is proved in `TapeStorageRates.native_joint_typical_partition` using the walk theorem of [2, Corollary O(a)].

For a fixed τ, record the values at Γ_τ and the input bits outside U_τ. Call this complete record z. Let $G_{\tau,z}$ be the inputs with that trace and record, and $C_{\tau,z}$ their projections onto U_τ. Fixing the exterior makes the projection injective. It also turns every omitted helper into a constant, so $C_{\tau,z}$ is a code for the induced relation on the retained ports. Each codeword extends to a member of $G_{\tau,z}$. These are the storage and extension statements of `TapeStorageRates.NativeTypicalCut.storage` and `TapeStorageRates.NativeTypicalCut.mem_code_iff`. The full trace fixes the successive append flags, hence the step at which each output position is written.

Proposition 6.1 (Counting native fibres)**.** Lean.
`TapeStorageRates.NativeTypicalPartition.deficit_moment`,
`TapeStorageRates.NativeTypicalPartition.tail_of_joint_count` in
`TapeStorageRates/NativeTypical.lean`.

Let X be uniform on G and use a native partition as above. For an attained cell define its deficit $D_{\tau,z} = m_\tau - \log_2 |C_{\tau,z}|$. Write D_X for the deficit of the cell containing X. For any real budget $\ell_{\tau,z}$ on each attained record, with ℓ_X the corresponding random budget,

$$\mathbb{E}\, 2^{D_X - \ell_X} = \frac{1}{|G|} \sum_{\tau} \sum_{z:\, G_{\tau,z} \neq \varnothing} 2^{m_\tau - \ell_{\tau,z}}.$$

If at most 2^h distinct pairs of trace and revealed step values occur on G, then for every $t \geq 0$,

$$\Pr[D_X > L - \log_2 |G| + h + t] \leq 2^{-t}.$$

Outline of the formal proof. The $|G_{\tau,z}|$ inputs in a cell all contribute the same power of two. Since $|C_{\tau,z}| = |G_{\tau,z}|$, this cardinality cancels when we sum over the cell, giving its term in the identity.

For the second display, use the constant budget $\ell = L - \log_2 |G| + h + t$. A fixed trace and fixed revealed word allow at most $2^{L - m_\tau}$ exterior assignments. Their combined contribution to the unnormalized sum is at most $2^{L-\ell}$, independently of m_τ. Summing over the attained pairs gives $\mathbb{E}\, 2^{D_X - \ell} \leq 2^{-t}$. On the event $D_X > \ell$ its integrand exceeds one. The same argument with the individual budgets bounds $\Pr[D_X > \ell_X]$ when the set of possible exterior assignments is known more precisely. □

6.2 A tail bound at fixed degree

Theorem 6.2 (Theorem N, typical native rate)**.** Lean.
`TapeStorageRates.typical_native_rate_tail`, `TapeStorageRates.typical_native_rates_full` in
`TapeStorageRates/NativeTypicalRates.lean`.

Fix M with k work tapes and $c \geq 0$. Put $p = 4k^2 + 1$ and $Z_\Delta = 2^{\sqrt{\log_2 \Delta}}$. There are constants $K, A > 0$ such that at every sufficiently large integer cap Δ one can fix an integer $s \geq 1$ with

$$f_p(\Delta) \leq 1/s \leq K f_p(\Delta).$$

For every fixed integer radius $r \geq 0$, all sufficiently large d, every $b \geq 1$ and every nonempty $G \subseteq G_{d,b}$, the bit simulator N_s has a partition of G with the following properties.

Projecting an attained cell as in Section 6.1 gives a storage code on the retained input positions of its trace. Every input and recovery port has degree at most Δ, every surviving paired distance exceeds r, and

$$m_\tau \geq (1 - Z_\Delta/\Delta) L.$$

The simulator halts within $(11c+1)(L+1)$ steps on G. For X uniform on G, let $R_X = \log_2 |C_{\tau,z}|/m_\tau$ for its cell. On this same partition, for every $u \geq 0$,

$$\Pr\left[R_X < \frac{\log_2 |G|}{L} - A f_p(\Delta) - u\right] \leq 2^{-uL}.$$

Outline of the formal proof. At fixed s the startup cost fits in $L+1$ steps for all sufficiently large d, uniformly in $b \geq 1$. The exact clock and trace count in Theorem 5.1 give the displayed time bound and at most $2^{aL/s}$ traces on G, for a constant $a = a(M, c)$. All inputs in G have the same output length.

On each trace, group the physical steps into the simulator's batches. A complete batch has $10s+1$ steps. Each head sweeps all cells in three aligned blocks and otherwise stays in those blocks. Replace its physical position by its block index. The simultaneous work-head trajectory admits nine coarse slots per complete batch and one for the final drain. This is still a trajectory of k unit-step work heads, with $O_c(L/s)$ vertices. Batch boundaries are retained, including when the heads stay in the same blocks.

Apply the full-path cut theorem to this coarse trajectory with cut fraction $\eta_s = \Theta_{M,c}(1/s)$ [2, Definition 7.1 and Corollary 9.3(a)]. Mark the whole batch containing each selected coarse vertex, and reveal every physical step record in the marked batches. A last-visit dependency between distinct batches lifts to a coarse last-visit chain confined to those endpoint batches: an intermediate complete batch visiting the same block would also visit the precise physical cell, contrary to the last-visit property. The chronological spine lifts as well. Counting path lengths in vertices, a coarse free-path bound H_s gives a physical bound $(10s+1)H_s+1$. The extra vertex allows a revealed root; the suffix of the path must avoid every marked batch. The path transfer is `TapeStorageRates.BatchCoarsening.Coarsening.path_length_le`.

We bound the joint support of these revelations using one local seed per marked batch. At entry to a marked batch, the full simulator control and the initial contents of its three-block input and work windows determine every step record in that batch. The control includes the buffered values and pending output. The seed uses $O_M(s)$ bits, so a set J of marked batches has at most $2^{O_M(s)|J|}$ joint record values; no independence is assumed. Since $|J| \le O_c(\eta_s L/s)$, choosing the constant in η_s makes this support at most $2^{L/s}$. We fix the complete records themselves. The seeds only bound their number. The joint support estimate is `TapeStorageRates.NativeBatchRecord.compiler_joint_record_support`.

The coarse cut parameter is now $x = 4/\eta_s = O_{M,c}(s)$. For the physical depth parameter D_s, put $B_s = \sum_{j=0}^{D_s}(k+1)^j$. The factor $10s+1$ in the lifted depth preserves the estimate

$$\log_2 \log_2(2sB_s) = O_{M,c}(s^p \log_2(s+1)).$$

The numerical argument is formalized in `TapeStorageRates.genie_scaled_cap_degree_bound_at`. Choose the constant in the inversion so that the double logarithm is at most $\frac{1}{2}\log_2\log_2\Delta$. Then $\max(s, B_s) \le Z_\Delta$ at the stated scale for s, by `TapeStorageRates.native_batch_size_for_cap`. This gives the same function f_p as the packet bound while retaining the physical bit positions and actual append roots.

There are at most $B_s L$ incidences. Removing input coordinates of degree greater than Δ therefore removes at most $B_s L/(\Delta+1)$ coordinates. In the induced graph, the short-pair estimate bounds the second deletion by $E_r L/\log_2 d$ for a constant E_r depending on the fixed parameters and radius, uniformly over the traces and the widths $b \ge 1$ [4, Proposition 4.11(a), accompanying Lean theorem]. Since $B_s/(\Delta+1) < Z_\Delta/\Delta$, both deletions fit in the claimed fraction at all sufficiently large d. Deleting coordinates can only increase the remaining paired distances.

Put $\lambda = Z_\Delta/\Delta$. For large caps $Z_\Delta^2 \le \Delta$, so $\lambda \le 1/s$. The chosen traces and genie words jointly cost at most $(a+1)L/s$ bits. Apply Proposition 6.1 with $t = uL$. Outside an event of probability at most 2^{-uL} we obtain

$$\log_2 |C_{\tau,z}| \ge \log_2 |G| - (\lambda + (a+1)/s + u)L.$$

The code is nonempty, so its rate on $m_\tau \le L$ coordinates is at least $\log_2|C_{\tau,z}|/L$. The loss $\lambda + (a+1)/s$ is at most $(a+2)Kf_p(\Delta)$. This proves the tail with a single partition for every confidence level. □

For a full successful cube $\log_2|G|/L = 1$. Taking $u = 1/s$ gives rate $1 - O_{M,c}(1/s)$ except on input mass at most $2^{-L/s}$. For a smaller success set its own rate enters with coefficient one. The

retained positions and recovery relation may vary from trace to trace. The code can also vary with the revealed word and the exterior values. The native coding target has the same rate-loss scale as the packet target, while keeping the physical positions and append times available for a more specific upper bound. Establishing such an upper bound remains open.

6.3 A sufficient native coding bound

The native half-rate hypothesis concerns the full cone relation of a specified machine at a specified depth. It asks for an upper bound $2^{(1/2+\delta)L}$ on every binary storage code on each eligible cone relation for all sufficiently large d, for every fixed $\delta > 0$. The step width $\mathrm{blockWidth}(N_s, 1)$ is the number of bits charged for revealing one complete physical record.

Corollary 6.3 (The native frozen-trace criterion)**.** `Lean.` `TapeStorageRates.native_frozen_trace_exclusion`, `TapeStorageRates.native_tests_exclude_linear` in `TapeStorageRates/TraceTransfer.lean`.

For each M and $c > 0$ one can choose a single aligned simulator N_s and time coefficient $c' > 0$ with the following implication. Suppose the native half-rate hypothesis of [2, Corollary L(b)] holds at canonical widths for N_s at scale one, time coefficient c', and depth parameter $\mathrm{genieDepth}(k, \mathrm{blockWidth}(N_s, 1), \max(c', 1), 1/8)$. *Then M succeeds on the full cube at only finitely many canonical dimensions. If this hypothesis holds on the chosen simulator for each source machine and coefficient,* (T) *is false.*

Outline of the formal proof. An eligible run halts by the specified time bound. The hypothesis ranges over cuts with the prescribed remaining depth and all choices of times before halt at the prescribed output-counter positions. In particular it includes the actual append steps. Choose s so that the complete trace support costs at most $L/4$ bits at large successful dimensions, using `TapeStorageRates.aligned_low_trace_dimensions`, which supplies $c' = 11c + 1$. A largest trace contains at least $2^{3L/4}$ inputs. Revealing at most $L/8$ bits on that trace leaves a common word with at least $2^{5L/8}$ inputs. These inputs form a code on the untrimmed append-time cones by the same predecessor induction as in Section 2. This exceeds the hypothesized bound for every $0 < \delta < 1/8$. The formal criterion applies the skeleton-entropy lower bound of Corollary L(b). Skeleton entropy is at most the logarithm of the full trace support. The separate witness theorem `TapeStorageRates.native_five_eighths` constructs the native code and `TapeStorageRates.NativeFibreWitness.outputTime_eq_appendTime` identifies its roots. □

The same checked criterion allows any fixed $0 < \eta < 1/2$. Choose the simulator so that its full trace support costs at most ηL bits at large successful dimensions. In the depth parameter, replace the cut fraction $1/8$ by $(1/2 - \eta)/2$. The common trace and genie fibre then has rate at least $1 - \eta - (1/2 - \eta)/2 > 1/2$.

7 What can overlapping patches predict?

A bit that is zero with probability 0.9 can be predicted with 90% accuracy without reading any helper. The information supplied by an allowed helper set B is $I(X_q; X_B) = H(X_q) - H(X_q \mid X_B)$. In the prediction credit

$$1 - H(X_q \mid X_B) = [1 - H(X_q)] + I(X_q; X_B),$$

the first term is due to the marginal bias and the second measures the gain from observing the helpers. The geometry of the relation bounds the sum of these gains, even when different patches

use the same bits. Write $X_S = (X_v)_{v \in S}$ and use entropy H in bits. The Shannon entropy in the Lean declarations is measured in nats, so its value in bits is obtained by division by $\ln 2$.

Let V be a finite set of coordinates. Each member of a finite indexed family of *patches* $P_j \subseteq V$ has a set $A_j \subseteq P_j$ of scored targets. For each $q \in A_j$, choose the helpers $B_{jq} \subseteq P_j$ available to its predictor. A target scored in two patches contributes two occurrences (j, q). Form the bipartite graph on the two copies of A_j by joining L_v to R_q when $v \in B_{jq} \cap A_j$. A patch satisfies the *separation condition* if L_q and R_q lie in different components of this graph for every $q \in A_j$.

An ambient distance bound gives a sufficient separation test. Take a relation N. Suppose $B_{jq} \subseteq N(q) \cap P_j$, $|A_j| \leq w$ with $w \geq 1$, and $\operatorname{dist}(L_q, R_q) > 2w - 1$ for every scored occurrence. A connection inside the patch would contain a simple path on at most $2|A_j|$ ports. The same path in the ambient graph would have length at most $2w - 1$. Thus the separation condition holds. The patch can contain arbitrarily many unscored helpers.

Theorem 7.1 (Theorem P, information supplied by overlapping patches)**.** `Lean`. `TapeStorageRates.weighted_patch_capacity`, `TapeStorageRates.patch_capacity_of_long_targets` in `TapeStorageRates/PatchResidual.lean`.

Let h be a polymatroid rank on V, that is, a normalized, monotone, submodular set function. Write $h(q \mid B) = h(B \cup \{q\}) - h(B)$. Give each coordinate a capacity $c_q \geq h(\{q\})$ and each patch a weight $\lambda_j \geq 0$. Suppose the separation condition holds, and $\sum_{j : v \in P_j} \lambda_j \leq r$ for every coordinate v. Then

$$\sum_j \lambda_j \sum_{q \in A_j} (c_q - h(q \mid B_{jq})) \leq r \left[\left(\sum_{v \in V} c_v - h(V) \right) + \left(\sum_{v \in V} h(\{v\}) - h(V) \right) \right].$$

For a finite random vector X, write $\mathrm{TC}(X) = \sum_v H(X_v) - H(X_V)$ for its total correlation. Taking $h(S) = H(X_S)$ and $c_v = H(X_v)$ gives

$$\sum_j \lambda_j \sum_{q \in A_j} I(X_q; X_{B_{jq}}) \leq 2r \, \mathrm{TC}(X).$$

Outline of the formal proof. Fix one patch and write $P = P_j$, $A = A_j$, $D = P \setminus A$. Condition on D. All remaining helpers of a target q have their left ports in the component of R_q. Order the components and first supply the targets whose left component precedes their right component. Then process the other targets in order of their left components. For each such target its right component came earlier, so all of its helpers are available. A supplied target costs at most $h(q \mid D)$ in the chain rule, and a target reached with its helpers costs at most $h(q \mid B_{jq})$. Reversing the order exchanges the two roles for each target. Adding the bounds gives

$$2\big(h(P) - h(D)\big) \leq \sum_{q \in A} h(q \mid D) + \sum_{q \in A} h(q \mid B_{jq}).$$

The formal proof groups the targets by their left components to obtain a single elimination piece in the sense of [5]. The common-observations corollary of the soft-recovery memo gives this inequality [9, Corollary 3.2]. The two component orders explain the same factor two as in the exact reconstruction argument of [7, Lemma 4.1].

Use $h(q \mid D) \leq h(\{q\})$ and bound $h(D)$ both by $\sum_{v \in D} h(\{v\})$ and by $\sum_{v \in D} c_v$. The contribution of this patch is at most the sum of the two deficits on the right side of the theorem, evaluated on P. To sum either deficit, let a_v stand for c_v or $h(\{v\})$ and fix an order on V, with $V_{<v}$ its initial segment before v. Diminishing returns gives

$$\sum_{v \in P} a_v - h(P) \leq \sum_{v \in P} (a_v - h(v \mid V_{<v})).$$

On weighting and summing, each nonnegative term on the right has coefficient at most r. The chain rule on V gives the claim. The formal summation in `TapeStorageRates.patch_deficit_packing` uses the weighted submodular cover inequality from the Lean library of [8]. □

Corollary 7.2 (Errors of local predictors)**.** Lean. `TapeStorageRates.PatchPredictor.prediction_capacity`, `TapeStorageRates.PatchPredictor.prediction_deficit` in `TapeStorageRates/PatchPrediction.lean`.

Let X be a random binary word on $n = |V|$ coordinates. Use unit patch weights and the ambient distance test above. Let r bound the number of patches containing any coordinate. Each occurrence $i = (j, q)$ uses a fixed Boolean predictor of X_q from $X_{B_{jq}}$ with error probability e_i. Assume the number m of scored occurrences is positive. Set $\bar{e} = m^{-1} \sum_i e_i$, and $\delta = n - H(X_V)$. With $h_2(t) = -t \log_2 t - (1-t) \log_2(1-t)$ and $0 \log_2 0 = 0$,

$$m[1 - h_2(\bar{e})] \le \sum_i [1 - h_2(e_i)] \le r(\delta + \mathrm{TC}(X)) \le 2r\delta.$$

Outline of the formal proof. The predictor and its error bit together determine X_q. Thus $H(X_q \mid X_{B_{jq}}) \le h_2(e_i)$, so $c_q - h_2(e_i)$ is a lower bound on the contribution of occurrence i in Theorem 7.1 for every choice of the capacities. Use $c_v = 1$ in Theorem 7.1 and $H(X_v) \le 1$. Concavity of binary entropy gives the bound in terms of $\bar{e}$. □

Equality holds at every error level $0 \le e \le 1$. In one patch, score both coordinates of $(U, U \oplus Z)$, where U is a fair bit and Z is an independent bit with $\Pr(Z = 1) = e$. Each predictor copies its partner. Both paired distances are infinite, $r = 1$, both errors equal e, and the joint entropy is $1 + h_2(e)$. Thus the two sides of the bound $m[1 - h_2(\bar{e})] \le 2r\delta$ equal $2[1 - h_2(e)]$. For $e \ne 1/2$ this proves that the factor two is necessary for general binary laws. The example is `TapeStorageRates.noisy_pair_prediction_sharp`.

For a uniform word in a binary code of length $n > 0$, suppose $\delta/n \le D/s$ with $D, s > 0$. If $m \ge \alpha n$, then

$$r \ \ge \ \frac{\alpha[1 - h_2(\bar{e})]}{2D}\, s.$$

This is `TapeStorageRates.PatchPredictor.overlap_of_near_unit_rate`. Predicting a positive density of separated targets with a fixed advantage over a fair guess therefore requires overlap growing at least linearly in s. For the families constructed in Theorem 3.1, take the radius $2w - 1$ after fixing s and w. For a native fibre supplied by Theorem 6.2, X can be taken uniform on that particular fibre, using its own actual permitted helpers and entropy deficit.

Corollary 7.3 (Exact recovery over a finite alphabet)**.** Lean. `TapeStorageRates.recovery_patch_coverage` in `TapeStorageRates/Patches.lean`.

Let $C \subseteq \Sigma^V$ be nonempty, where $a = |\Sigma| \ge 2$. Each patch P_j serves at most $w \ge 1$ targets in $A_j \subseteq P_j$. For each $q \in A_j$, its value on C is determined by the coordinates in $N(q) \cap P_j$. Each coordinate belongs to at most r patches. Let $S \subseteq \bigcup_j A_j$, and let $b_N(t)$ count all coordinates in V at paired distance at most t. Then

$$|S| \le b_N(2w - 1) + 2r(n - \log_a |C|).$$

In fact the last term alone bounds the number of distinct targets in S at distance greater than $2w - 1$.

Outline of the formal proof. Give C the uniform law. In each patch score only the served targets at distance greater than $2w-1$. Their conditional entropies given the permitted helpers vanish. In Theorem 7.1 take $c_v = \log_2 a$. Each long occurrence contributes $\log_2 a$, and every distinct long target in S contributes at least once. Both global deficits are at most $n \log_2 a - \log_2 |C|$. □

Thus even if the alphabet varies with the code, a family with a vanishing fraction of short pairs at radius $2w-1$ has upper limiting covered fraction at most $2r(1-\liminf_i R_i)$, where $R_i = \log_{|\Sigma_i|} |C_i|/|V_i|$ and w, r are fixed. This is `TapeStorageRates.patch_coverage_limsup`. Disjoint repeated pairs attain the finite bound at $r = 1$, as in `TapeStorageRates.repetition_pairs_sharp`.

Corollary 7.4 (Input-dependent choice of predictors)**.** `Lean.`
`TapeStorageRates.patch_prediction_menu_probability`,
`TapeStorageRates.patch_prediction_event_probability` in
`TapeStorageRates/PatchPredictionEvent.lean`.
Let $E \subseteq \{0,1\}^V$, $n = |V|$, $0 \le e \le 1/2$, and $r \ge 1$. Fix a menu of at most 2^h predictor systems satisfying the ambient distance test with common bounds w, r. A system specifies its relation, patches, scored targets, helpers and prediction rules. Suppose that, for each $x \in E$, some system scores at least m occurrences and errs on at most a fraction e of all the occurrences it scores on x. Then

$$\frac{|E|}{2^n} \le 2^{h - m[1-h_2(e)]/(2r)}.$$

In particular a transcript selecting the system can be charged $h = \log_2 |\{\text{labels attained on } E\}|$ when $E \neq \varnothing$.

Outline of the formal proof. For each system, restrict to the inputs of E on which it meets the two requirements. Under the uniform law on any such nonempty subset, its mean error is at most e. By Corollary 7.2 and monotonicity of h_2 on $[0, 1/2]$, the subset has at most $2^{n - m[1-h_2(e)]/(2r)}$ elements. Their union contains E. □

A tape lower bound along this route still requires a proof that successful executions admit predictors with the stated accuracy and overlap. On a fixed native fibre one can test the predictors under its uniform law. On the full cube one must also bound the number of distinct systems selected by the trace, the genie word and any other observation used to choose the targets.

8 What remains open

How close to one can the abstract rate get? Known binary constructions exceed 0.5013 at a fixed degree cap. For larger caps the large-prime construction gives guaranteed rates arbitrarily close to $4 - 2\sqrt{3}$ from below [6, Theorem A*, Lemma 8.1 and Corollary A′]. Both facts leave room for the rate upper bound in Corollary 4.1. A proof along this route must control all admissible families at the caps used in the test. A concrete construction target for a given tape count k is a family of limiting lower rate at least $1 - K f_{4k^2+1}(\Delta)$ at each sufficiently large cap, with one constant K. Such families would preclude the gap in Corollary 4.1 for that k. A tape algorithm would also have to realize their recoveries by one causal computation within its running-time budget.

Which helpers can a physical computation use? Theorem 6.2 keeps the time at which each retained bit is appended. This allows a prospective impossibility proof to choose patches from a tape schedule and test them on its actual code. Fix a positive prediction advantage ξ and a target density $\alpha > 0$. For each simulator batch size s, choose a per-patch target bound w_s before the

dimension grows and use paired-distance radius $2w_s - 1$. Consider cells with the following property. Under the uniform law on that cell, one fixed system scores at least an α fraction of its retained coordinates and predicts them from permitted helpers with mean accuracy at least $1/2 + \xi$. Its overlap is bounded by a common $r_s = o(s)$. Must such cells carry a fraction of the uniform input mass bounded away from zero as the dimension grows?

For a full successful cube at fixed s, the cells of deficit fraction $O(1/s)$ carry all but at most $2^{-L/s}$ of the input mass. A positive mass bound independent of the dimension would give a cell meeting both conditions. On that cell Corollary 7.2 would require overlap of order s, contradicting the proposed upper bound at large s. If targets or predictors are chosen separately for each input, the menu test of Corollary 7.4 charges the number of systems used. Establishing a prediction advantage on actual runs remains open. Two independent fair helpers can determine their XOR exactly even though either one alone gives chance accuracy.

A Formal statements and verification

The accompanying Lean development is authoritative for the numbered statements and their proofs. The human-readable outlines give the mathematical arguments and explain the hypotheses. Inputs, moves, output and halting use the first paper's frozen machine model [3].

Packet coordinates. `TapeStorageRates.PaidInputs.GrowingWidthSuccess` quantifies over a common choice of dimension and entry width for each size threshold and accuracy tolerance. The source machine supplies its own class of successful inputs. `TapeStorageRates.PaidClasses.compiler` realizes the eleven-tick batch computation. `TapeStorageRates.packet_code_at_threshold` performs the finite extraction and `TapeStorageRates.family_of_packet_classes_at_threshold` chooses dimensions to give a family. The positive lengths, two-sided degree bounds and paired distances belong to `TapeStorageRates.SeparatedBinaryFamily`. The comparison with the supremum of limiting upper rates is `TapeStorageRates.RateWitnesses.extremal_rates`.

Native coordinates. `TapeStorageRates.NativeTypicalPartition` chooses one cut and retained coordinate set per attained trace. Each input restricts to a word of its own cell code. The membership proof is `TapeStorageRates.NativeTypicalPartition.projected_mem`. The tail theorem concerns the uniform probability on G under these assignments. For Theorem 6.2, depth reduction is applied to the coarse work walks; the joint support of complete batch records pays for the physical revelations. Corollary 6.3 instead uses the native coding hypothesis of the fourteenth paper at its specified physical step width and depth [2, Corollary L].

Predictions. `TapeStorageRates.PatchPredictor.missed` masks all inputs outside a predictor's permitted helpers before evaluating its guess. The development uses natural logarithms for Shannon entropy. Dividing by $\ln 2$ gives the bit units of Section 7. Counting one scored occurrence for each patch and target allows one target to occur in several patches. The finite-alphabet exact-coverage result counts distinct covered targets.

Corollary A.1 (The original qualitative target)**.** Lean. `TapeStorageRates.qualitative_transfer` in `TapeStorageRates/Main.lean`.

Under (T)*, for each* $\varepsilon > 0$ *and every sufficiently large cap there is an admissible family of limiting lower rate at least* $1 - \varepsilon$*.*

Outline of the formal proof. A witness to (T) succeeds at all canonical dimensions. The original target uses $\max(1, k)$ in place of the actual tape count k. The quantitative statement is proved by `TapeStorageRates.quantitative_transfer`. Its qualitative consequence follows by `TapeStorageRates.qualitative_of_quantitative`. At that fixed tape count the deficit scale tends to zero. Theorem 3.1 also gives the bound with k itself, including $k = 0$. □

Reproducing the checks. Use the complete repository so the sibling Lean packages are available. The project pins Lean and Mathlib v4.34.1. From this paper's directory run

```
(cd lean && lake build && lake env lean PaperAxioms.lean)
python3 tools/lean_index.py --check
```

The generated axiom file queries the declarations cited in the text and in the formal status lines. Their dependencies use at most `propext`, `Classical.choice` and `Quot.sound`. The finite phase calculations use locally increased heartbeat limits; their proof terms are checked by the same kernel. The README gives the preparation steps for a fresh checkout.

B Index of Lean declarations

The status lines of the paper cite the following declarations, listed by Lean module with their fully qualified names. Module files are relative to the paper's `lean/` directory. The axiom checks in `lean/PaperAxioms.lean` are generated from the same sources.

`TapeStorageRates.Main` `TapeStorageRates.exponentially_small_at_widths_of_sparse_gap`, `TapeStorageRates.qualitative_transfer`, `TapeStorageRates.rate_witnesses_of_full`, `TapeStorageRates.rate_witnesses_of_growing_widths`, `TapeStorageRates.rate_witnesses_of_near_full`.

`TapeStorageRates.NativeCompiler` `TapeStorageRates.Native.same_partial_function`, `TapeStorageRates.Native.source_support`, `TapeStorageRates.Native.source_support_fixed`.

`TapeStorageRates.NativeTypical` `TapeStorageRates.NativeTypicalPartition.deficit_moment`, `TapeStorageRates.NativeTypicalPartition.tail_of_joint_count`.

`TapeStorageRates.NativeTypicalRates` `TapeStorageRates.typical_native_rate_tail`, `TapeStorageRates.typical_native_rates_full`.

`TapeStorageRates.PacketFamily` `TapeStorageRates.packet_code_at_threshold`.

`TapeStorageRates.PatchPrediction` `TapeStorageRates.PatchPredictor.prediction_capacity`, `TapeStorageRates.PatchPredictor.prediction_deficit`.

`TapeStorageRates.PatchPredictionEvent`
`TapeStorageRates.patch_prediction_event_probability`, `TapeStorageRates.patch_prediction_menu_probability`.

TapeStorageRates.PatchResidual TapeStorageRates.patch_capacity_of_long_targets, TapeStorageRates.weighted_patch_capacity.

TapeStorageRates.Patches TapeStorageRates.recovery_patch_coverage.

TapeStorageRates.TraceTransfer TapeStorageRates.native_frozen_trace_exclusion, TapeStorageRates.native_tests_exclude_linear.

Also named in the text From this paper's library:
TapeStorageRates.BatchCoarsening.Coarsening.path_length_le,
TapeStorageRates.Native.control_bits, TapeStorageRates.Native.copy_trace_card,
TapeStorageRates.Native.simulates_halt, TapeStorageRates.Native.trace_support_rate_limsup,
TapeStorageRates.NativeBatchRecord.compiler_joint_record_support,
TapeStorageRates.NativeFibreWitness.outputTime_eq_appendTime,
TapeStorageRates.NativeTypicalCut.mem_code_iff, TapeStorageRates.NativeTypicalCut.storage,
TapeStorageRates.NativeTypicalPartition,
TapeStorageRates.NativeTypicalPartition.projected_mem,
TapeStorageRates.PaidClasses.compiler, TapeStorageRates.PaidInputs.GrowingWidthSuccess,
TapeStorageRates.PatchPredictor.missed,
TapeStorageRates.PatchPredictor.overlap_of_near_unit_rate,
TapeStorageRates.Prefix.control_bits, TapeStorageRates.Prefix.simulates_halt,
TapeStorageRates.Prefix.source_support, TapeStorageRates.Prefix.time_le_eleven,
TapeStorageRates.RateWitnesses.extremal_rates, TapeStorageRates.SeparatedBinaryFamily,
TapeStorageRates.aligned_low_trace_dimensions,
TapeStorageRates.elementary_gap_ratio_tendsto_zero,
TapeStorageRates.family_of_packet_classes_at_threshold,
TapeStorageRates.genie_scaled_cap_degree_bound_at,
TapeStorageRates.native_batch_size_for_cap, TapeStorageRates.native_five_eighths,
TapeStorageRates.native_joint_typical_partition,
TapeStorageRates.noisy_pair_prediction_sharp, TapeStorageRates.patch_coverage_limsup,
TapeStorageRates.patch_deficit_packing, TapeStorageRates.qualitative_of_quantitative,
TapeStorageRates.quantitative_transfer, TapeStorageRates.ratio_tendsto_of_power_gap,
TapeStorageRates.repetition_pairs_sharp.
From Paper 11's library: MergeDepth.skel_paired_packing.

References

[1] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded nesting and depth reduction for matrix transposition on tapes. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, October 2026.

[2] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded-stack DAGs reduce to constant depth. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, November 2026.

[3] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, October 2026.

[4] Hans Brende and Claude Opus 5.5 (Anthropic). Merge depth, input steering, and the coding core of matrix transposition on tapes. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, October 2026.

[5] Hans Brende and Claude Opus 5.5 (Anthropic). Nested elimination and half-rate bounds for storage codes. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, October 2026.

[6] Hans Brende and Claude Opus 5.5 (Anthropic). Storage codes above half rate under paired-distance separation. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, November 2026.

[7] Hans Brende and GPT-6 Astra (OpenAI). Directed recovery structures in tape computations. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, October 2026.

[8] Hans Brende and GPT-6 Astra (OpenAI). Local coding barriers and the limits of tape geometry for matrix transposition. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, October 2026.

[9] Hans Brende and GPT-6 Astra (OpenAI). Soft recovery and record capacities in nested elimination. Manuscript with Lean sources, HansBrende/papers, revision 5ea6e23c9c4a, October 2026.

[10] Ohad Elishco and Alexander Barg. Recoverable systems. *IEEE Transactions on Information Theory*, 68(6):3681–3699, 2022. doi:10.1109/TIT.2022.3153330.

[11] J. Hartmanis and R. E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965. doi:10.1090/S0002-9947-1965-0170805-7.

[12] John Hopcroft, Wolfgang Paul, and Leslie Valiant. On time versus space. *Journal of the ACM*, 24(2):332–337, 1977. doi:10.1145/322003.322015.

[13] R. Ryan Williams. Simulating time with square-root space. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 13–23, 2025. URL: https://people.csail.mit.edu/rrw/time-vs-space.pdf, doi:10.1145/3717823.3718225.